

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Y. Ramadevi	Reg. No.:	192373065
Date:	17/08/2026	Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Git & Docker Technical Assignment

Containerized Full-Stack Task Management and Performance Monitoring System

Develop a full-stack web-based Task Management and Performance Monitoring System that enables users to create, assign, update, prioritize, and monitor tasks and projects through an interactive web interface.

The system shall use React for the frontend and Node.js with Express.js for backend REST APIs. A database shall be used to store users, projects, tasks, deadlines, priorities, and task-status information.

The application shall provide a performance monitoring module that analyzes task completion, pending tasks, deadlines, priorities, and individual/team performance. Efficient searching, sorting, filtering, and task-prioritization algorithms shall be implemented to improve application performance.

The complete project shall be managed using Git and GitHub with an appropriate branching strategy, meaningful commits, merging, and collaborative development practices. The frontend and backend shall be containerized using Docker, while Docker Compose shall be used to manage multiple services and their communication.

The system should demonstrate how version control and containerization improve collaboration, portability, maintainability, deployment, scalability, and reliability of a full-stack application.

1. Problem Analysis

1.1 Problem Description

Traditional task management systems may provide basic task creation and tracking facilities but often lack integrated performance monitoring, efficient task prioritization, collaborative development, and portable deployment environments.

The proposed system provides a centralized platform where users can manage tasks and projects while monitoring progress and performance.

The application consists of:

- User management
- Project management
- Task creation and assignment
- Task status tracking
- Priority management
- Deadline monitoring
- Search and filtering
- Performance monitoring
- Task completion statistics
- Git-based version control
- Docker-based deployment

1.2 Objectives

The main objectives are:

1. To develop a full-stack task management application using React and Node.js.
2. To provide REST APIs for task and project management.
3. To implement efficient task searching, sorting, and prioritization.
4. To monitor user and team task performance.
5. To manage source code using Git and GitHub.
6. To implement a suitable Git branching strategy.
7. To containerize application components using Docker.
8. To manage multiple containers using Docker Compose.
9. To improve application portability and deployment consistency.
10. To demonstrate scalable and maintainable software development practices.

1.3 Functional Requirements

The system should provide:

- User registration and login
- Project creation
- Task creation
- Task assignment
- Task editing and deletion
- Task priority selection
- Task status updates
- Deadline management
- Task searching
- Task filtering and sorting
- Performance dashboard
- Completed/pending task statistics
- User-wise performance analysis
- REST API communication
- Database storage
- Containerized application deployment

1.4 Expected Outcomes

The expected outcome is a functional full-stack application in which users can manage tasks and monitor performance through a web interface.

The project should also demonstrate:

- Proper Git repository management
- Feature-based branching
- Meaningful commit history
- Docker containerization
- Docker Compose configuration
- Successful container-to-container communication
- Application deployment and testing

2. Project Structure Design

The proposed repository structure is:

```
task-management-system/
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   └── services/
│   ├── App.js
│   ├── index.js
│   ├── package.json
│   └── Dockerfile
├── backend/
│   ├── controllers/
│   ├── models/
│   ├── routes/
│   ├── middleware/
│   ├── config/
│   ├── server.js
│   ├── package.json
│   └── Dockerfile
├── database/
│   └── init/
├── docs/
│   ├── architecture.png
│   └── workflow.png
├── .gitignore
├── docker-compose.yml
└── README.md
```

Folder / File	Purpose
frontend/	Contains the React front-end application
public/	Static files and public assets
src/	Source code of the front-end application
components/	Reusable UI components
pages/	Page components / Screens
services/	API calls and service functions
App.js, index.js	Main React application files
package.json	Dependencies and scripts for front-end
Dockerfile	Docker instructions for front-end
backend/	Contains the Node.js + Express backend
controllers/	Request handlers and business logic
models/	Database models / schemas
routes/	API route definitions
middleware/	Authentication and custom middleware
config/	Configuration files (DB, env, etc.)
server.js	Entry point of the backend server
package.json	Dependencies and scripts for backend
Dockerfile	Docker instructions for backend
database/init/	Database initialization scripts
docs/	Project documentation and diagrams
architecture.png	System architecture diagram
workflow.png	Git & Docker workflow diagram
.gitignore	Files and folders ignored by Git
docker-compose.yml	Docker Compose configuration for all services
README.md	Project summary, setup and usage instructions

Purpose of Major Components

Folder/File	Purpose
frontend/	Contains the React application
components/	Reusable UI components
pages/	Application screens
services/	API communication logic
backend/	Node.js/Express server
controllers/	Business logic
models/	Database models

routes/	REST API routes
middleware/	Authentication and request processing
config/	Database and application configuration
database/	Database initialization files
docs/	Architecture and workflow diagrams
Dockerfile	Instructions for creating Docker images
docker-compose.yml	Defines and manages multiple containers
.gitignore	Specifies files excluded from Git
README.md	Project documentation

3. Git Repository Initialization

The project repository is initialized using Git.

Git Commands

Steps to Initialize Git Repository

- Create a project folder
`mkdir task-management-system && cd task-management-system`
- Initialize a Git repository
`git init`
- Add files to staging area
`git add .`
- Commit the initial changes
`git commit -m "Initial project setup"`
- Rename the branch to main
`git branch -M main`
- Connect to remote repository (GitHub)
`git remote add origin https://github.com/your-username/task-management-system.git`
- Push code to remote repository
`git push -u origin main`

Commands Used (Example)

```
PS D:\Projects> mkdir task-management-system
PS D:\Projects> cd task-management-system
PS D:\Projects\task-management-system> git init
Initialized empty Git repository in D:/Projects/task-management-system/.git/
PS D:\Projects\task-management-system> git add .
PS D:\Projects\task-management-system> git commit -m "Initial project setup"
[main (root-commit) 1b2e3d] Initial project setup
5 files changed, 120 insertions(+)
create mode 100644 README.md
create mode 100644 .gitignore
create mode 100644 docker-compose.yml
create mode 100644 frontend/Dockerfile
create mode 100644 backend/Dockerfile
PS D:\Projects\task-management-system> git branch -M main
PS D:\Projects\task-management-system> git remote add origin https://github.com/your-username/task-management-system.git
PS D:\Projects\task-management-system> git push -u origin main
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 8 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (15/15), 3.21 KiB | 3.21 MiB/s, done.
Total 15 (delta 0), reused 0 (delta 0)
To https://github.com/your-username/task-management-system.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Purpose of Important Git Files

- .git/**
Contains all Git metadata, objects, history, branches, configuration, etc.
- .gitignore**
Specifies files and folders that Git should ignore (node_modules, .env, logs, etc.)
- README.md**
Contains project overview, usage instructions and documentation.
- docker-compose.yml**
Defines and runs multi-container Docker applications.
- Dockerfile**
Contains instructions to build Docker images.

.gitignore Example

```
node_modules/
.env
dist/
build/
*.log
npm-debug.log*
Docker *.db
.DS_Store
```

These files and folders will not be tracked by Git.

GitHub Repository (After Push)

your-username / task-management-system Public

1 Branch 0 Tags

File	Commit	Time
frontend	Initial project setup	now
backend	Initial project setup	now
docs	Initial project setup	now
.gitignore	Initial project setup	now
docker-compose.yml	Initial project setup	now
README.md	Initial project setup	now

Folder Structure (After Initialization)

```
task-management-system/
├── frontend/
├── backend/
├── docs/
├── .gitignore
├── docker-compose.yml
└── README.md
```

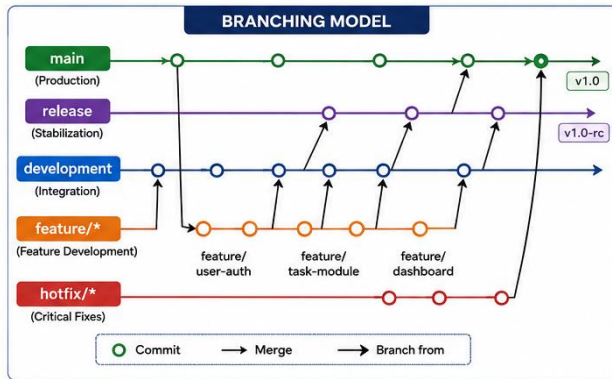
The repository is successfully initialized and ready for development and collaboration.

To check status at any time:
`git status`

Git provides a complete history of changes and allows developers to collaborate safely.

4. Git Branching Strategy

A feature-based branching strategy is selected for the project.



BRANCHES EXPLAINED				
Branch	Purpose	Created From	Merged To	Description
main	Production-ready code	-	-	Contains stable and released versions of the application.
development	Integration branch for ongoing development	main	main, release	All feature branches are merged here first. Tested before release.
feature/*	Develop a specific feature	development	development	Used for new features, improvements or modules. One feature per branch.
release	Prepare for production release	development	main, development	Stabilization branch for testing and bug fixes before final release.
hotfix/*	Fix critical issues in production	main	main, development	Used only for urgent bug fixes in production environment.

- BRANCHING WORKFLOW**
- Create a feature branch from **development**.
 - Develop the feature and commit changes.
 - Push the feature branch to remote repository.
 - Create a Pull Request to **development**.
 - Code review and merge after approval.
 - Test in **development** branch.
 - Create a **release** branch from **development**.
 - After testing, merge **release** to **main** and tag the version.
 - If any critical issue found in production, create **hotfix/*** branch from **main**, fix and merge to **main** and **development**.

EXAMPLE GIT COMMANDS

```

# Create and switch to development branch
git checkout main
git pull origin main
git checkout -b development
git push -u origin development

# Create a feature branch
git checkout development
git checkout -b feature/task-module
git push -u origin feature/task-module

# After development & commit, go back and merge
git checkout development
git pull origin development
git merge feature/task-module
git push origin development

# Create release branch
git checkout -b release/v1.0 development
git push -u origin release/v1.0

# Merge release to main after testing
git checkout main
git merge release/v1.0
git tag -a v1.0 -m "Version 1.0 Release"
git push origin main --tags

# Hotfix from main
git checkout -b hotfix/login-bug main
# (fix the issue)
git add . && git commit -m "Fix login bug"
git checkout main
git merge hotfix/login-bug
git checkout development
git merge hotfix/login-bug
git push origin main development
  
```

- BENEFITS OF THIS STRATEGY**
- Keeps main branch stable and production-ready.
 - Encourages parallel development.
 - Easy to manage features, releases, and hotfixes.
 - Reduces conflicts by isolating work.
 - Improves collaboration and code quality.
 - Provides a clear workflow from development → release → production.

- BEST PRACTICES**
- Always create a new branch for a new feature or fix.
 - Keep feature branches small and focused.
 - Write meaningful commit messages.
 - Do code review before merging.
 - Delete merged branches to keep the repo clean.
 - Regularly sync with remote repository.

Branches

Main:

Contains stable and production-ready code.

Development:

Contains integrated development work before release.

Feature branches:

Used for individual features such as:

feature/frontend

feature/backend-api

feature/task-management

feature/performance-dashboard

feature/docker

Release:

Used to prepare a stable version of the application.

Hotfix:

Used to fix critical problems in the production version.

Justification

This branching strategy allows different developers to work on separate features without affecting the stable code. It also makes code reviews, testing, merging, and release management easier.

5. Version Control Workflow

The development workflow is:

Create Issue



Create Feature Branch



Develop Feature



Test Feature



git add



git commit



git push



Pull Request



Code Review



Merge to Development



Testing



Merge to Main

Important Git Commands

git status

git checkout -b feature/task-management

git add .

git commit -m "Add task creation and assignment"

git push origin feature/task-management

git checkout development

git pull origin development

git merge feature/task-management

git push origin development

Conflict Resolution

If a merge conflict occurs:

git pull origin development

Resolve the conflicting files manually and then:

git add .

git commit -m "Resolve merge conflict"

git push origin development

6. Commit History Analysis

Meaningful commits should be maintained throughout the project.

Example:

Initial project setup

Create React frontend

Add Express backend

Implement task management API

Add task database model

Implement task dashboard

Add performance monitoring

Add Docker configuration

Add Docker Compose

Fix API communication issue

Update project documentation

Good commit messages should:

- Be short and meaningful
- Describe the actual change
- Avoid unnecessary information
- Follow a consistent naming style

Example:

Good: Add task priority API

Good: Fix task filtering issue

Good: Add Docker configuration

Poor: changes

Poor: update

Poor: final

The commit history provides traceability and helps developers identify when and why changes were introduced.

7. Docker Environment Design

Docker is used to provide a consistent environment for developing, testing, and deploying the application.

Components to be Containerized

The proposed system contains:

React Frontend

|

Node.js/Express Backend

|

Database

Each major component can run inside its own container.

Benefits of Docker

- Consistent development environment
- Easy deployment
- Application isolation
- Portability
- Dependency management
- Simplified testing
- Easy scaling
- Reduced environment-related errors

8. Dockerfile Development

Frontend Dockerfile

```
FROM node:20

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm", "start"]
```

Dockerfile Instructions

Instruction	Purpose
FROM	Specifies the base image
WORKDIR	Sets the working directory
COPY	Copies application files
RUN	Executes commands during image creation
EXPOSE	Specifies the application port
CMD	Defines the default application command

Backend Dockerfile

```
FROM node:20

WORKDIR /app
```

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 5000

CMD ["npm", "start"]

The backend container runs the Node.js/Express REST API.

9. Docker Compose Design

Docker Compose is used to manage multiple services.

Example:

services:

frontend:

build: ./frontend

ports:

- "3000:3000"

depends_on:

- backend

backend:

build: ./backend

ports:

- "5000:5000"

depends_on:

- database

database:

image: mongo:7

ports:

- "27017:27017"

volumes:

- mongodb_data:/data/db

volumes:

mongodb_data:

Services

Frontend:

Runs the React application.

Backend:

Runs Node.js/Express APIs.

Database:

Stores users, tasks, projects, and performance information.

Networking

Docker Compose creates a network that allows containers to communicate using service names.

For example:

frontend → backend → database

Volumes

The database uses a volume to preserve data even if the database container is restarted.

10. Container Deployment and Testing

Build and start the application using:

```
docker compose up --build
```

To run the containers in the background:

```
docker compose up -d
```

To view running containers:

```
docker ps
```

To view logs:

```
docker compose logs
```

To stop the application:

docker compose down

Testing Procedure

1. Start Docker Desktop.
2. Open the project directory.
3. Execute docker compose up --build.
4. Verify that all containers are running.
5. Open the React frontend.
6. Test user and task operations.
7. Verify backend API responses.
8. Verify that data is stored in the database.
9. Check performance dashboard results.
10. Capture screenshots as evidence.

Suggested Screenshots

- GitHub repository
 - Git branch list
 - Git commit history
 - Pull request/code review
 - Docker build
 - docker ps
 - Docker Compose execution
 - Frontend application
 - Backend API response
 - Performance dashboard
-

11. Benefits of Git and Docker

Benefits of Git

Git provides:

- Version control
- Collaboration
- Branch management
- Code review
- Change tracking
- Conflict management
- Rollback capability
- Complete project history

Benefits of Docker

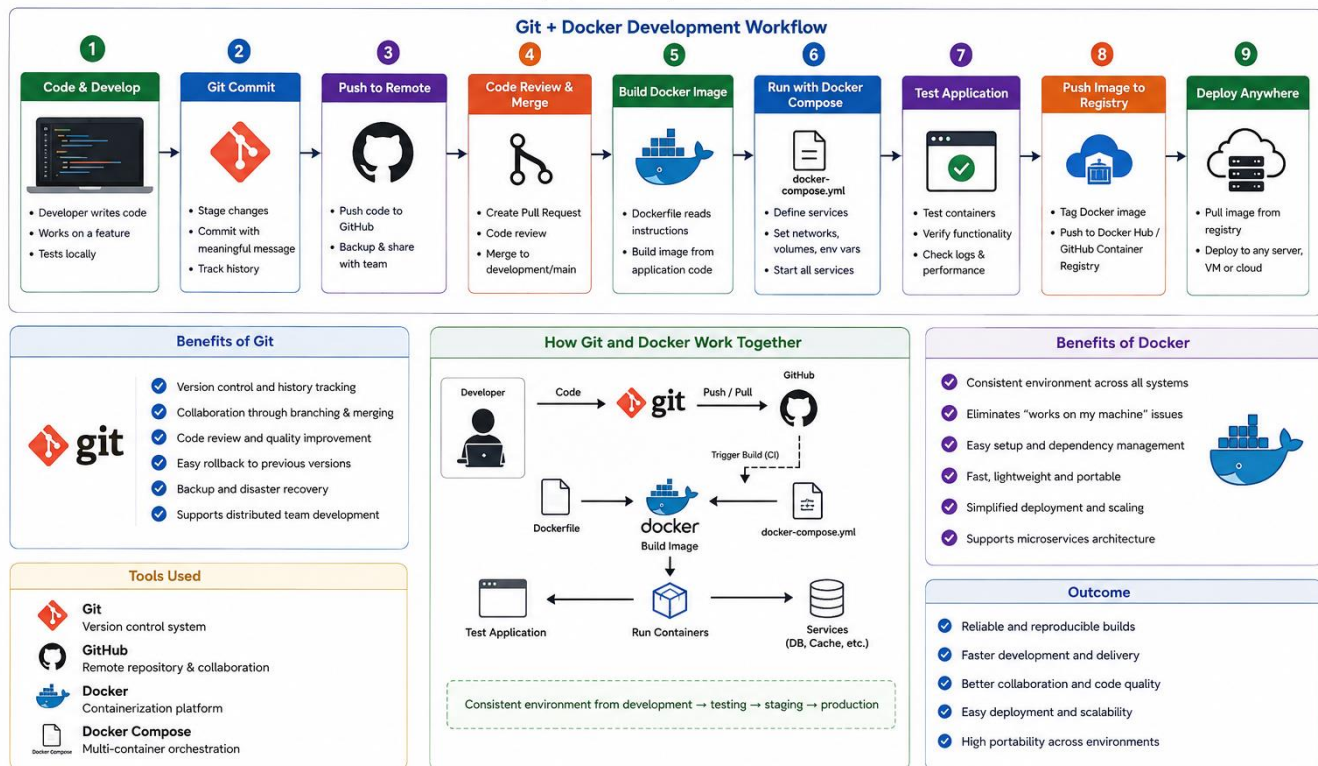
Docker provides:

- Portability
- Environment consistency
- Application isolation
- Simplified deployment
- Dependency management
- Scalability

- Reproducibility

Combined Benefits

Using Git and Docker together provides a reliable development workflow:



12. Challenges and Improvements

Challenges

Possible challenges include:

1. Managing dependencies between frontend and backend.
2. Configuring Docker networking.
3. Managing environment variables.
4. Resolving Git merge conflicts.
5. Maintaining a clean Git history.
6. Handling database persistence.
7. Debugging container-related errors.
8. Managing communication between containers.
9. Maintaining consistent development environments.
10. Optimizing application performance.

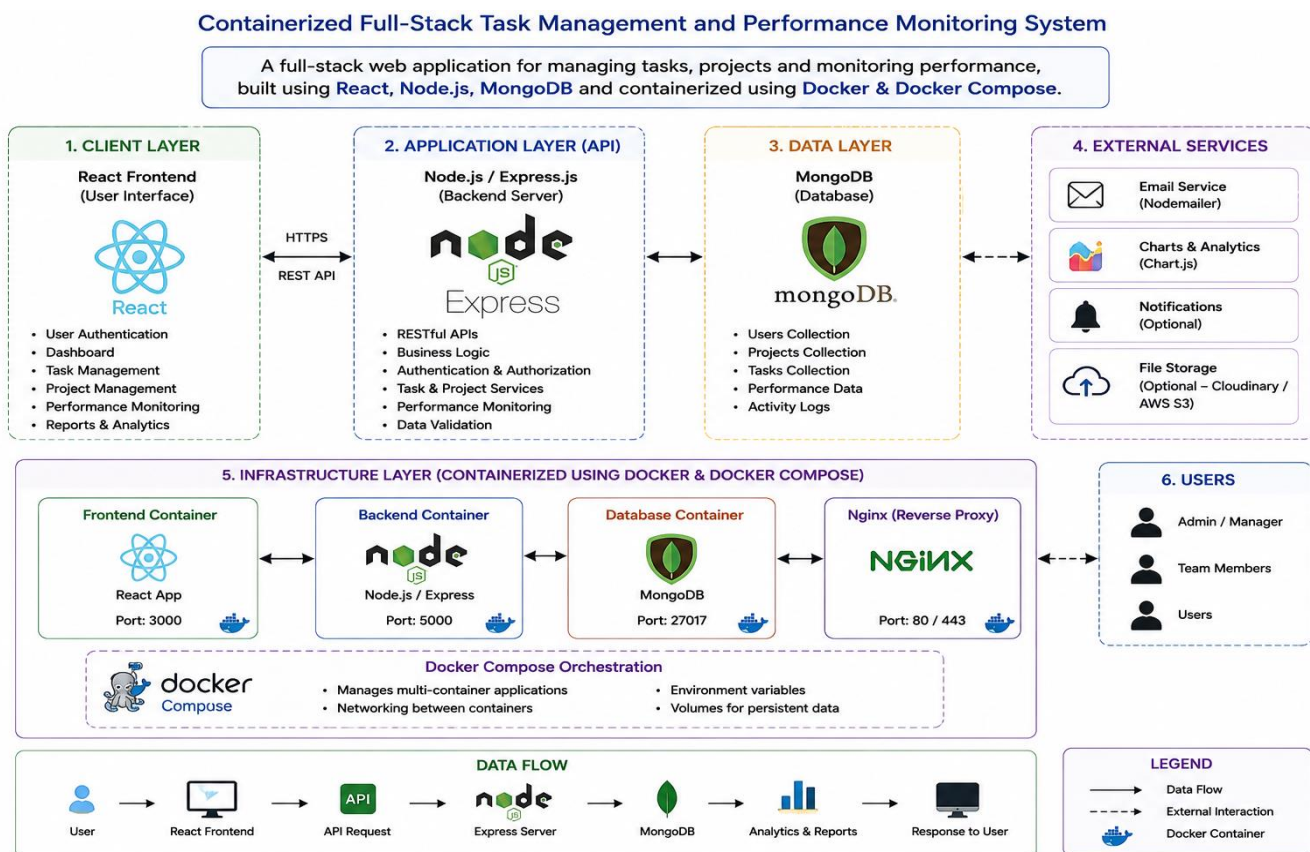
Future Enhancements

The system can be improved by implementing:

- Kubernetes-based deployment

- Helm charts
- CI/CD pipelines
- Automated testing
- Authentication and authorization
- Real-time notifications
- Advanced analytics
- Distributed task processing
- Load balancing
- Cloud deployment
- Application monitoring
- Automated Docker image publishing

Architecture Diagram

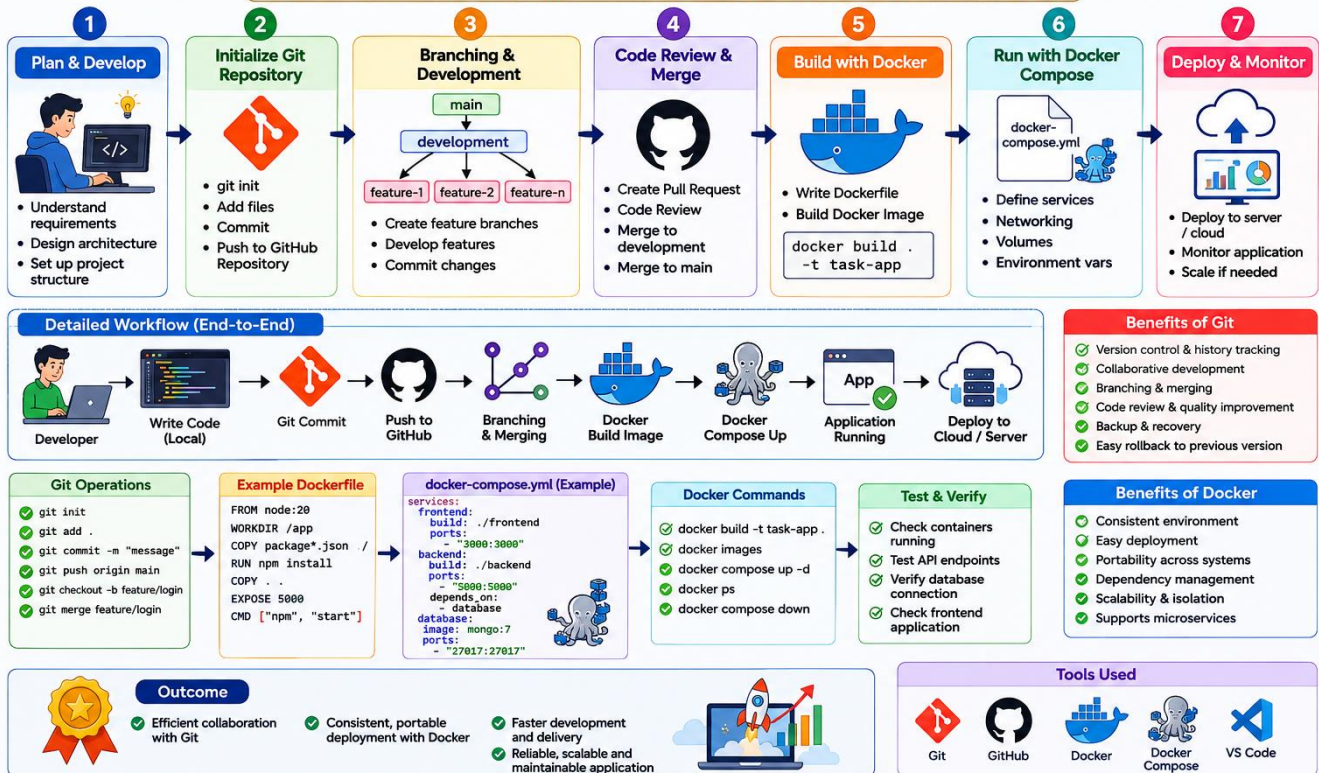


Git and Docker Workflow Diagram

Git and Docker Workflow Diagram

Reliable Development, Collaboration, Containerization and Deployment

Project: **Containerized Full-Stack Task Management and Performance Monitoring System**



System Requirement Analysis

Hardware Requirements

- Processor: Intel Core i3 or equivalent
- RAM: Minimum 4 GB
- Storage: Minimum 10 GB free space
- Internet connection for GitHub and Docker image retrieval

Software Requirements

- Windows/Linux/macOS
- Visual Studio Code
- Node.js
- npm
- React
- Node.js and Express.js
- MongoDB
- Git
- GitHub account
- Docker Desktop
- Docker Compose
- Web browser

Selection of Software Architecture

A three-tier architecture is selected.

Presentation Layer



Business/API Layer



Data Layer

Presentation Layer

React provides the user interface for task management and performance monitoring.

Business/API Layer

Node.js and Express.js process requests and provide REST APIs.

Data Layer

MongoDB stores users, projects, tasks, deadlines, priorities, and performance data.

This architecture provides separation of concerns, maintainability, scalability, and easier testing.

Component Interaction

The major components interact as follows:

User



React Frontend



REST API



Express Backend



Business Logic

↓

MongoDB

↓

Performance Analysis

↓

React Dashboard

The frontend sends requests to the backend using REST APIs. The backend validates and processes the requests, interacts with the database, and returns the required data to the frontend.

Quality Attribute Analysis

Quality Attribute	Implementation
Performance	Efficient searching, sorting and filtering
Scalability	Docker containers and future Kubernetes deployment
Maintainability	Modular frontend/backend structure
Portability	Docker-based deployment
Reliability	Git history and containerized environments
Security	Environment variables and authentication
Availability	Container restart and scalable deployment
Usability	Responsive React interface

Architectural Justification

The selected architecture is appropriate because the system requires a clear separation between the user interface, business logic, and database.

React provides an interactive frontend, while Node.js and Express.js provide lightweight REST APIs. MongoDB provides flexible storage for task and project data.

Docker ensures that each application component runs in a consistent environment. Git and GitHub provide version control and collaborative development.

The architecture can also be extended in the future with Kubernetes, Helm, CI/CD, load balancing, and cloud deployment, making it suitable for larger-scale applications.